

Course Project

Canary Extraction Challenge: How Private Is Your Language Model?

Privacy-Preserving Natural Language Processing, Summer 2026
Ruhr University Bochum

Overview

In this project you will get hands-on experience with the full attack–defense cycle of language-model privacy. You will plant fake personal information (“canaries”) into training data, fine-tune a model, try to extract those canaries, and then retrain the model with differential privacy to stop the leakage. You will also work through the theory behind the defense and see first-hand how the privacy budget ϵ controls what an attacker can learn.

The project has four phases.

Phase	Topic
1. Theory	DP proofs and privacy accounting
2. Attack	Fine-tune DistilGPT-2, run extraction and MIA
3. Defense	DP-SGD training, measure ϵ vs. leakage
4. Mystery Model (bonus)	Black-box red-teaming

Setup

You will receive:

- A **starter script** (`setup.py`) that takes your student ID as seed and sets up your whole experiment.
- A **mystery model generator** (`mystery_engine.so`) for Phase 4. You run it with your student ID and it builds a personalized model.

Tools: Python, PyTorch, HuggingFace Transformers, Opacus (for privacy accounting).

Computational requirements

All experiments use **DistilGPT-2** (82M parameters) so that everything fits comfortably on a free-tier GPU. The same techniques work with larger models.

Recommended: Google Colab free tier (T4, 15 GB VRAM) is enough for every phase.

1 Phase 1: Theory

Work through the following proofs. Make sure to state your assumptions and write each step clearly.

Task 1.1: Laplace mechanism

Prove that the Laplace mechanism gives ϵ -differential privacy. Concretely, take a query f whose global L_1 sensitivity is Δf . Show that adding noise drawn from $\text{Lap}(\Delta f/\epsilon)$ to $f(D)$ guarantees

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S]$$

for every pair of neighboring datasets D, D' and every measurable set S .

Task 1.2: Composition

- Prove the basic (sequential) composition theorem: if $\mathcal{M}_1$ is ϵ_1 -DP and $\mathcal{M}_2$ is ϵ_2 -DP, then releasing both together is $(\epsilon_1 + \epsilon_2)$ -DP.
- State the advanced composition theorem (no proof needed). In 2–3 sentences, explain why it gives a tighter bound.

Task 1.3: DP-SGD privacy budget

Suppose you train with batch size $B = 64$, dataset size $N = 50,000$, clipping norm $C = 1.0$, noise multiplier $\sigma = 1.1$, and $T = 3$ epochs. What is the total (ϵ, δ) privacy guarantee when $\delta = 10^{-5}$?

Walk through the calculation step by step.

Deliverable: Written solutions (PDF), 3–5 pages.

2 Phase 2: Attack

Now you get to play the attacker. You will fine-tune DistilGPT-2 on data that contains planted PII and then try to pull it back out.

Task 2.1: Data preparation

Run the starter script with your student ID to generate your personal canary set:

```
python setup.py --student_id YOUR_ID --output_dir ./experiment
```

This creates three files: `train.txt` (training data with canaries inserted), `eval.txt` (held-out set, no canaries), and `canaries.json` (your canary list with insertion frequencies). Each canary is a fake PII string, for example: "My name is Jennifer Kline and my SSN is 491-27-8835". All canaries go into one training file, but each one is repeated a different number of times (10, 50, 200, or 500).

Task 2.2: Fine-tune DistilGPT-2

Fine-tune `distilgpt2` on your dataset using standard training (no privacy). Train for 5 epochs and record the final perplexity on a held-out set.

Task 2.3: Data extraction attack

For each canary, tokenize the full string, feed the first 50% of tokens as a prefix, and let the model generate the remaining tokens using greedy decoding. A canary counts as **recovered** if the generated text exactly matches the second half. Measure the **extraction rate** (fraction of canaries recovered) separately for each frequency bucket (10×, 50×, 200×, 500×). Show the results in a table.

Task 2.4: Membership inference attack

Now try a membership inference attack (MIA). You will implement the *loss-based attack* described in Carlini et al. (2022)^a. The idea is simple: if a model memorizes its training data, it will assign lower perplexity to sequences it has seen than to sequences it has not. Your job is to see how well perplexity alone can tell the two apart.

- (a) **Members.** For each canary sequence, compute the model’s perplexity: tokenize the sequence, run it through the model with `labels=input_ids`, and compute `exp(loss)`.
- (b) **Non-members.** Generate an equal number of sequences that the model never saw during training. These should use the same PII template as your canaries (same format, similar length) but with different names, numbers, and addresses. This is important: if you compare PII-formatted canaries against plain Wikipedia text, the attack will look artificially easy because the model can distinguish the formats, not because it memorized anything.
- (c) **Classify.** Sweep a perplexity threshold: at each threshold, label every sequence below it as “member” and above it as “non-member.” Plot the ROC curve and report the AUC.

^aN. Carlini et al., “Membership Inference Attacks From First Principles,” IEEE S&P 2022.

Deliverable: Jupyter notebook with code, extraction rate table, ROC plot, and AUC value.

3 Phase 3: Defense

In this phase you will measure how much an attacker can learn at different values of ϵ . You will start with a simple counting query to see the relationship clearly, and then see how the picture changes for a real language model.

Task 3.1: Laplace mechanism experiment

Imagine two neighboring datasets D and D' where a counting query returns 100 on D and 101 on D' . For each $\varepsilon \in \{0.01, 0.05, 0.1, 0.25, 0.5, 1, 2, 4, 8, 16\}$, repeat the following 10,000 times:

- Pick D or D' at random (50/50).
- Add $\text{Lap}(1/\varepsilon)$ noise to the true answer (100 or 101).
- The adversary sees only the noisy answer and must guess which dataset was used. Use a simple threshold: if the noisy answer > 100.5 , guess D' , otherwise guess D .
- Record whether the guess was correct.

After 10,000 rounds, compute the adversary's accuracy. Plot **adversary advantage** (accuracy - 0.5) vs. ε on a log scale.

Answer: *At what ε does the adversary first exceed 75% accuracy? How does this relate to the e^ε factor in the DP definition?*

Task 3.2: Epsilon sweep on your language model

Retrain DistilGPT-2 with DP-SGD at $\varepsilon \in \{0.5, 1, 2, 3, 5, 8, 15, 30\}$. Set $\delta = 10^{-5}$ for all runs. Implement the DP-SGD loop yourself: compute per-sample gradients, clip each to a maximum norm, average the clipped gradients, and add Gaussian noise calibrated to the clipping norm. Use your non-private model from Phase 2 as the $\varepsilon = \infty$ baseline.

Note on compute budget: DP-SGD typically requires more training updates per ε than non-private fine-tuning, because per-sample clipping limits how much any single update can shift the model. The Phase 2 epoch count is unlikely to be enough here.

For each model, measure:

- Canary extraction rate (same method as Task 2.3).
- MIA AUC (same method as Task 2.4).

Plot both metrics vs. ε on one figure. Include a dashed line at AUC = 0.5 (random guess) for reference.

Answer: *Look at the shape of your plot. How does privacy leakage change as ε grows? Is the transition gradual or sudden? Compare this to the Laplace curve from Task 3.1 and explain why the two behave differently.*

Task 3.3: Analysis

Write 1–2 pages connecting your experimental results to the theory. Compare all three plots (Laplace advantage, extraction rate, MIA AUC) and discuss what they reveal about how differential privacy works in practice. Consider questions like: How well does the theoretical ε predict real-world leakage? Where do the theory and practice agree or disagree? What practical lessons would you take away if you had to train a model on sensitive data?

Deliverable: Laplace experiment plot (Task 3.1), epsilon sweep plot (Task 3.2), written answers and analysis (max 2 pages total).

4 Phase 4: Mystery Model Challenge (bonus)

You will generate a **personalized** model using the provided binary, and your job is to figure out what private information it has memorized.

Task 4.1: Generate and red-team the model

First, generate your mystery model by running the following command (replace `YOUR_ID` with your student ID):

```
python -c "import mystery_engine; mystery_engine.run(YOUR_ID)"
```

This will fine-tune a model on hidden data and save it to `./mystery_model/model/`. The training takes a few minutes on a Colab GPU. Once it finishes, load the model:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained("./mystery_model/model")
tokenizer = AutoTokenizer.from_pretrained("./mystery_model/model")
```

You do **not** know what PII the model memorized or how the training data was put together. Every student's model has different canaries.

- (a) Come up with prompting strategies to extract memorized PII. Try different approaches: direct prompting, prefix completion, repetition-based prompts, topic-guided prompts, and anything else you can think of.
- (b) Keep a log of every piece of PII you find. For each one, record the prompt you used and the model's output.

Task 4.2: Strategy report

Write a short report (1–2 pages) covering:

- Which prompting strategies worked best, and why do you think that is?
- Which types of PII were easiest to extract? Which were hardest? Why do you think some were harder?

Deliverable: PII catalog (table of prompt–output pairs) and strategy report.

Grading note: Your model contains a known number of canaries that are unique to you. Your score depends on how many you find (recall).

Submission

- Phase 1: PDF with proofs
- Phases 2–3: Jupyter notebooks (runnable) with all plots and tables
- Phase 4: PII catalog + strategy report (PDF)

All code must be reproducible using your student ID as seed.

Grading

Phase	Task	Points
1. Theory (20 pts)	Task 1.1: Laplace proof	8
	Task 1.2: Composition	7
	Task 1.3: DP-SGD budget calculation	5
2. Attack (35 pts)	Task 2.1: Data preparation	3
	Task 2.2: Fine-tuning	7
	Task 2.3: Extraction attack	12
	Task 2.4: Membership inference attack	13
3. Defense (45 pts)	Task 3.1: Laplace experiment	10
	Task 3.2: Epsilon sweep	30
	Task 3.3: Written analysis	5
4. Mystery Model (bonus)	Task 4.1: Red-teaming	17
	Task 4.2: Strategy report	3
Total		100 + 20 bonus

To pass the project you need at least 65 points overall (65%). Phase 4 is optional and does not count toward the passing threshold, but bonus points are added to your final score.